

Lab 7 — Capstone: The Sensor Pipeline

ESP32-S3 FIRMWARE LAB SERIES · PHASE 1 · FOUNDATIONS · CAPSTONE

Estimated time	3–4 hrs
Combines	Labs 2, 4, 5
Prerequisites	Labs 2, 4 and 5 — this lab combines all three.
Beyond the board	The same IMU wiring as Lab 5. Nothing new.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§7.1 objective**, **§7.2 parts and wiring**, **§7.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

7.1 Objective & TM4C bridge

Sample the IMU at a fixed 1 kHz, accumulate a window of samples, and compute features over each window. This is the general **acquire → process → decide → communicate** skeleton, and it is reusable: whatever you build on top of it, the decide stage (a threshold, a control law, or a small model) becomes just another consumer of this feature stream rather than a rewrite.

TM4C BRIDGE

You have probably built this exact thing before. The standard bare-metal ADC lab is: sample an analog input, convert the reading, display it on an HD44780 LCD. That is **acquire → process → output**, the same three stages. What changes here is not the goal but the *structure*. On the TM4C all three lived in one `while(1)` loop paced by `delayMs()`, and if the LCD write was slow the next sample was simply late. Here the stages are separate tasks running at different rates, handing off through kernel objects, so a slow consumer can never delay the sampler. That structural difference is the entire point of this lab.

7.2 Parts & wiring

Nothing new. This lab reuses the Lab 5 wiring exactly: IMU on 3V3, GND, SDA to GPIO8, SCL to GPIO9. If the Lab 5 rig is still on the bench, you are already wired. See Lab 5 §5.2 if you took it apart.

One optional addition worth making: leave a **spare GPIO free for instrumentation**. In §7.7 you will toggle a pin at the top and bottom of the sampler and watch it on the analyzer to measure real execution time and jitter. GPIO4 is free in this lab and is a good choice.

7.3 How it works

What problem is this lab actually solving?

Start from the naive version, because you would write it first and it is genuinely instructive to see it fail. One task, one loop:

```
// the naive version -- do not use, but understand why
while (1) {
    read_accel();           // ~0.3 ms over I2C at 400 kHz
    store_sample();
    if (buffer_full) {
        compute_features(); // several ms of floating-point math
        printf(...);       // several MORE ms, serial is slow
    }
    vTaskDelay(1);         // try to pace at 1 kHz
}
```

Three things are wrong, and each one teaches a real lesson.

1. The sample rate is not actually 1 kHz. `vTaskDelay(1)` is one *tick*, and the ESP-IDF default tick is 100 Hz, so that is a 10 ms delay — 100 Hz, not 1000 Hz. Even with the tick raised, delay is *relative*: it sleeps one tick *after the work finishes*, so the true period is (work time + delay) and it drifts with however long the work took. This is the exact drift problem Lab 2 introduced `vTaskDelayUntil` to fix, and the reason Lab 4 introduced a hardware timer.

2. The slow work blocks the fast work. Every 256th iteration this loop stops to do milliseconds of math and serial printing. During that stall, no sampling happens at all. Your 1 kHz stream develops a hole every quarter second — and the hole is invisible in the output, because the buffer still fills up, just with samples that are not evenly spaced in time. Any frequency analysis of that data is silently wrong. **This is the failure that motivates the whole lab.**

3. There is no way to change one rate without disturbing the other. The sampling rate and the reporting rate are welded together by the loop structure.

The fix is to separate the stages so each runs at its own rate, and connect them with something that carries a signal across contexts without either side polling. That is what the rest of this chapter is about.

Why 1 kHz, and not 10 kHz

The sample rate is not arbitrary and it is not a CPU limit — the chip runs at 240 MHz and is nowhere near busy here. Two other things set the ceiling.

The sensor's own update rate. The MPU-6500's accelerometer produces new data at a configured output rate; reading faster than that just returns the same value again, so you gain nothing but bus traffic. Its internal digital low-pass filter also bandwidth-limits the signal, so there is a genuine physical limit to what the part can tell you.

I²C bus bandwidth. This is usually the real constraint. At 400 kHz, one six-byte register read costs roughly 60 bit-times of address and register overhead plus 54 more for the data and ACKs — call it 200 μ s of bus time in round numbers once you include start and stop conditions. At 1 kHz you have a 1000 μ s budget per sample, so you are using a fifth of it and have comfortable headroom. Push to 5 kHz and the budget drops to 200 μ s, which is the entire transaction — you would be saturating the bus with no margin for jitter.

That reasoning is worth having ready. "Why 1 kHz?" answered with *"because the bus transaction is about 200 μ s and I wanted five times headroom"* is an engineering answer; "because the manual said so" is not. And if you genuinely needed 10 kHz, the fix is not a faster CPU — it is SPI instead of I²C, or the sensor's own FIFO read in batches.

The three contexts

Lab 7 has three pieces of code that run independently, and understanding *who runs what, and how often* is most of the lab.

Context	Runs	Does	Must be
Timer callback <code>tick_cb</code>	1000×/sec, driven by hardware	Exactly one thing: give the semaphore	Tiny. No I ² C, no printing, no blocking.
Sampler task (prio 6)	1000×/sec, woken by the semaphore	Read the IMU, compute magnitude, store into the window	Fast and predictable. This is the real-time path.
Feature task (prio 5)	~4×/sec, woken when a window fills	Mean, RMS, peak over 256 samples; log the result	Allowed to be slow. It is off the critical path.

The rates differ by a factor of 250, and that is the point: **each stage runs at the rate its job actually requires**, instead of everything being dragged to the speed of the slowest step.

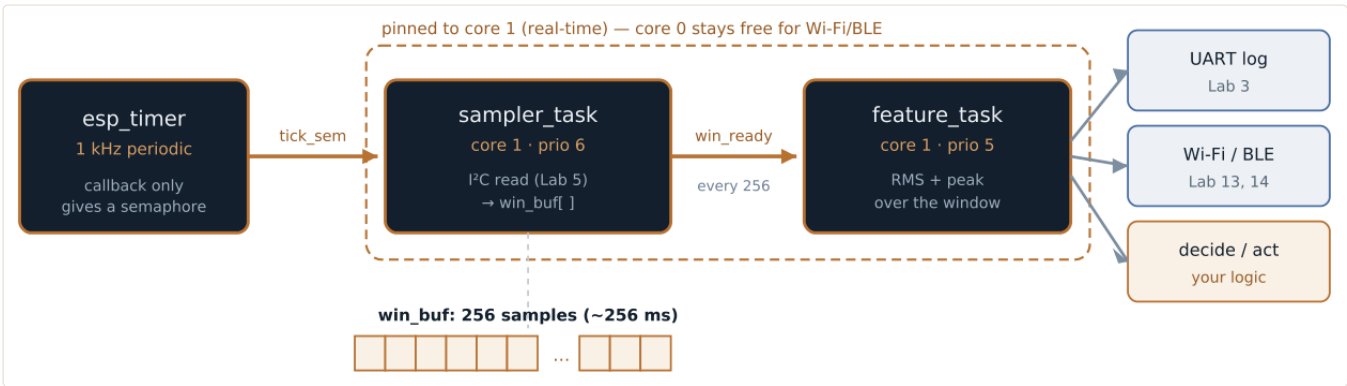


Figure 7.1. A general sensor-node architecture — the shape most embedded projects take. Every arrow is a FreeRTOS primitive. Hang whatever machinery you like off the right-hand outputs; the "decide / act" box is where your specific flagship logic goes.

Sensor node architecture: the acquire / process / act pipeline.

Flow: `esp_timer` (1 kHz periodic; the callback does nothing but give a semaphore) --tick_sem--> `sampler_task` (core 1, priority 6; does the I2C read; writes into `win_buf[]`) --win_ready, every 256 samples--> `feature_task` (core 1, priority 5; computes RMS and peak over the window) --> outputs.

Outputs branch three ways: UART log (Lab 3), Wi-Fi / BLE (Labs 13, 14), and "decide / act" which is your own application logic.

Both tasks are pinned to core 1 (the realtime side) so core 0 stays free for the Wi-Fi and BLE stacks.

`win_buf` holds 256 samples, which at 1 kHz is about 256 ms of signal.

Semaphores, properly

A semaphore is the piece most people nod along to without really getting, so here it is from the ground up.

The problem it solves. Task A finishes something; task B needs to know. On bare metal you would write a global flag and have B spin on it: `while (!flag) { }`. That works and it is what you would have done on the TM4C. But it burns 100% of a CPU core doing nothing, and under an RTOS it is actively harmful — a spinning task at decent priority can starve everything below it.

What a semaphore is. A kernel object holding a count, with two operations. **Give** increments it. **Take** decrements it — but if the count is already zero, the taking task is **blocked**: moved out of the ready list entirely, consuming zero CPU, until someone gives. The scheduler then wakes it immediately. Same logical handshake as the flag, but the waiting is free and the wake-up is instant.

Binary versus counting. A **binary** semaphore's count saturates at 1 — it is a flag that is either raised or not. Two gives before a take are indistinguishable from one give; the second is lost. A **counting** semaphore counts up to a configured maximum, so N gives allow N takes. Lab 7 uses binary, which has a consequence worth naming: if the sampler ever takes longer than 1 ms, a tick is silently dropped rather than queued up. That is usually the *right* behaviour for a sampler (better to skip than to fall permanently behind), but it is a design decision, not an accident, and it is an excellent thing to say out loud in an interview.

Which primitive do I reach for?

This question comes up constantly and is a standard interview probe. The decision is about *what you are communicating*, not personal preference.

Use	When	Carries data?	Has an owner?	In this series
Binary semaphore	Signalling: "an event happened, go run"	No	No	Lab 7 — timer says "tick"
Counting semaphore	Signalling events that can pile up, or managing N identical resources	No	No	—
Queue	Passing <i>data</i> from one context to another, with buffering	Yes	No	Lab 8 — ISR sends which GPIO fired
Mutex	Protecting a shared resource so only one task uses it at a time	No	Yes	Lab 16 — the priority-inversion fix

The mutex row is the one that matters most. A mutex is *owned* by the task that took it, and only that task may give it back. That ownership is what enables **priority inheritance**: if a high-priority task blocks on a mutex held by a low-priority task, the kernel temporarily raises the holder's priority so it

can finish and release. A binary semaphore has no owner and therefore cannot do this — which is exactly the bug Lab 16 makes you reproduce. **Rule of thumb: signalling between contexts → semaphore or queue. Protecting shared state → mutex, always.**

Why the timer callback must stay tiny

The callback does one line of work. That is deliberate, and the reasoning generalises to every interrupt and callback you will ever write.

The callback runs in a privileged, time-critical context. While it runs, the thing it is delaying is *the next tick*. If the callback took 0.4 ms, you would have burned 40% of your 1 ms budget before the sampler even started. And an I²C read inside it would be far worse: `i2c_master_transmit_receive` **blocks** waiting on the bus, and blocking in a callback or ISR context is not merely slow, it is illegal — there is no task there to put to sleep.

So the pattern is universal: **the fast context captures the event, a normal task does the work**. In Lab 8 this same split has a formal name, the **top half** (ISR) and **bottom half** (task). Learn it here where it is simple, because Lab 8 tests it with a real interrupt.

TM4C bridge: in a bare-metal timer lab the ISR sets a flag and clears the interrupt in `ICR`, and `main()` polls that flag. Identical division of labour. The upgrade is that the RTOS lets the waiting side *sleep* instead of poll, and wakes it the instant the flag is set.

What a window is, and why 256

You cannot compute how much something is vibrating from a single sample — one number has no variation. Vibration is a property of a *stretch of time*, so you collect a block of consecutive samples and compute over that block. That block is a **window**.

Window length is a real engineering trade-off, and being able to discuss it is worth more than the code:

- **Longer window** → smoother, more statistically stable numbers, and the ability to resolve lower frequencies. But slower to react and more memory.
- **Shorter window** → snappier response to a change, less memory. But noisier numbers.

At 1 kHz, 256 samples is 256 ms of data and yields about 4 updates per second, which feels responsive to a human watching a log. 256 is also a power of two, which matters the moment you move to an FFT, because FFT implementations want power-of-two lengths. Choosing 256 rather than 250 costs nothing now and saves a rewrite later.

These windows are **non-overlapping**: fill 256, process, start fresh. Production DSP often uses *overlapping* windows (advance by 128, reuse the previous 128) to get more frequent updates without shortening the window. Not needed here, but that is the vocabulary.

What RMS and peak actually mean

Two numbers come out of each window. Both are standard vibration features, not arbitrary.

The code first computes the **mean** of the window and then measures everything relative to it. That subtraction matters: a board sitting still on the desk reads about 1.0 g of gravity, and gravity is a

constant offset (**DC**) that tells you nothing about vibration. Subtracting the mean removes it, leaving only the fluctuation (the **AC** part). This is why a still board gives RMS near zero rather than near one.

RMS (root mean square) is the square root of the average squared deviation. Physically it is the "typical" amount of movement across the whole window, and it is the standard measure of vibration energy because squaring makes negatives count the same as positives and weights big excursions more heavily. It is the number that rises steadily as a machine gets rougher.

Peak is the single largest deviation in the window. It catches a one-off impact that RMS would average away — a single sharp knock barely moves RMS but spikes peak.

Together they distinguish two genuinely different faults: *continuous roughness* (RMS up, peak moderate) versus *intermittent impacts* (RMS near normal, peak spiking). The ratio of the two, **crest factor**, is a real diagnostic used in industrial condition monitoring, and mentioning it signals you understand what the features are for.

Core pinning and priorities

Why pin to core 1. The chip has two cores. Wi-Fi and Bluetooth stacks run on core 0 and are bursty and time-sensitive. Pinning both of your tasks to core 1 keeps your deterministic sampling away from radio activity. This costs nothing now and prevents mysterious jitter later when Labs 13 and 14 switch the radio on. `xTaskCreatePinnedToCore` is `xTaskCreate` with one extra trailing argument for the core number.

Why sampler priority 6 and feature priority 5. If both become ready at the same instant, the higher number wins and runs first. The sampler is on the real-time path with a hard 1 ms deadline; the feature task can be a few milliseconds late with no consequence. So the sampler gets the higher priority. The general rule is **rate-monotonic**: the more frequent and more deadline-sensitive the task, the higher its priority. Getting this backwards — the slow reporting task preempting the sampler — would put holes back in your data and undo the whole design.

7.4 Procedure & code

1. Confirm the Lab 5 wiring is intact and that WHO_AM_I still answers.
2. Create the project, set the target, type the listing.
3. Flash, then watch the feature stream while you tap the desk.

```
#include <math.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "driver/i2c_master.h"
#include "esp_timer.h"
#include "esp_log.h"

#define SDA_PIN 8
#define SCL_PIN 9
#define MPU_ADDR 0x68
#define FS_HZ 1000 // sample rate
#define WIN 256 // samples per feature window (~256 ms)

static const char *TAG = "vib";
```

```

static i2c_master_dev_handle_t mpu;
static SemaphoreHandle_t tick_sem;    // timer -> sampler
static SemaphoreHandle_t win_ready;   // sampler -> feature
static float win_buf[WIN];
static int win_idx = 0;

static void tick_cb(void *arg) {
    xSemaphoreGive(tick_sem);        // tiny: just release the sampler
}

static void sampler_task(void *arg) {
    while (1) {
        xSemaphoreTake(tick_sem, portMAX_DELAY);
        uint8_t reg = 0x3B, r[6];
        i2c_master_transmit_receive(mpu, &reg, 1, r, 6, 1000);
        int16_t ax = (int16_t)((r[0] << 8) | r[1]);
        int16_t ay = (int16_t)((r[2] << 8) | r[3]);
        int16_t az = (int16_t)((r[4] << 8) | r[5]);
        float mag = sqrtf((float)ax*ax + (float)ay*ay + (float)az*az) / 16384.0f;
        win_buf[win_idx++] = mag;
        if (win_idx >= WIN) { win_idx = 0; xSemaphoreGive(win_ready); }
    }
}

static void feature_task(void *arg) {
    while (1) {
        xSemaphoreTake(win_ready, portMAX_DELAY);
        float mean = 0;
        for (int i = 0; i < WIN; i++) mean += win_buf[i];
        mean /= WIN;
        float ss = 0, peak = 0;
        for (int i = 0; i < WIN; i++) {
            float d = win_buf[i] - mean;
            ss += d * d;
            if (fabsf(d) > peak) peak = fabsf(d);
        }
        float rms = sqrtf(ss / WIN);    // AC RMS of the signal magnitude
        ESP_LOGI(TAG, "rms=%.4f peak=%.4f", rms, peak);
    }
}

void app_main(void) {
    // --- I2C bring-up ---
    i2c_master_bus_config_t bus_cfg = {
        .clk_source = I2C_CLK_SRC_DEFAULT, .i2c_port = I2C_NUM_0,
        .sda_io_num = SDA_PIN, .scl_io_num = SCL_PIN,
        .glitch_ignore_cnt = 7, .flags.enable_internal_pullup = true,
    };
    i2c_master_bus_handle_t bus;
    i2c_new_master_bus(&bus_cfg, &bus);
    i2c_device_config_t dev_cfg = {
        .dev_addr_length = I2C_ADDR_BIT_LEN_7,
        .device_address = MPU_ADDR, .scl_speed_hz = 400000,
    };
    i2c_master_bus_add_device(bus, &dev_cfg, &mpu);
    uint8_t wake[2] = { 0x6B, 0x00 };
    i2c_master_transmit(mpu, wake, 2, 1000);    // wake from sleep

    // --- plumbing ---

```

```

tick_sem = xSemaphoreCreateBinary();
win_ready = xSemaphoreCreateBinary();
// pin the real-time work to core 1; core 0 stays free for WiFi later
xTaskCreatePinnedToCore(sampler_task, "samp", 4096, NULL, 6, NULL, 1);
xTaskCreatePinnedToCore(feature_task, "feat", 4096, NULL, 5, NULL, 1);
const esp_timer_create_args_t a = { .callback = &tick_cb, .name = "fs" };
esp_timer_handle_t h;
esp_timer_create(&a, &h);
esp_timer_start_periodic(h, 1000000 / FS_HZ); // 1000 us = 1 kHz
}

```

7.5 Code walkthrough, line by line

Eighty-four lines. This lab combines the timer, the sensor and the RTOS, so the listing is the longest so far — everything is explained in full here, including the parts that also appeared in earlier labs.

Line 1 · #include

Standard C maths, needed for two functions: `sqrtf` (square root, float version) on lines 34 and 52, and `fabsf` (absolute value, float version) on line 50. The trailing `f` on both names matters — those are the `float` variants. The plain `sqrt` and `fabs` operate on `double`, which is 64-bit and needlessly slow on a 32-bit MCU whose floating-point unit is single-precision.

Lines 2-4 · FreeRTOS.h, task.h, semphr.h

`FreeRTOS.h` must come first: it defines the core types and the configuration the other FreeRTOS headers depend on. `task.h` brings the task creation and delay calls. `semphr.h` is new in this lab and brings semaphores — the type `SemaphoreHandle_t` and the give/take functions.

Line 5 · #include "driver/i2c_master.h"

ESP-IDF's modern I²C master driver, which provides the bus and device configuration structs and the transaction functions used on lines 59–72. Note there is an older deprecated API in `driver/i2c.h`; code found online often uses it and will not compile against this one.

Line 6 · #include "esp_timer.h"

The high-resolution timer API, used on lines 80–83 to fire the callback at a fixed rate. It lives under **System** rather than **Peripherals** in the documentation, because it is a software layer that multiplexes many timers onto one piece of hardware.

Line 7 · #include "esp_log.h"

ESP-IDF's logging system, home of the `ESP_LOGI` macro on line 53.

Lines 9-11 · #define SDA_PIN 8 / SCL_PIN 9 / MPU_ADDR 0x68

The two I²C wires and the sensor's bus address. **SDA** is the serial data line and **SCL** is the serial clock; both are shared by every device on the bus. `0x68` is the **bus address** — the 7-bit number that says *which chip* on those shared wires you are addressing. Be precise about what that physically is: it is not a bit pattern applied to pins, and it is not a memory location. It is seven bits **clocked out serially on the SDA wire**, one bit per SCL pulse, right after the start condition. Every device on the bus shifts those bits in and compares them against its own address; the one that matches responds and the rest

ignore the rest of the transaction. It is also unrelated to the register numbers used later (`0x3B` , `0x6B`) despite looking similar.

Line 12 · `#define FS_HZ 1000`

The sample rate, 1000 Hz. Named rather than typed inline because it appears again on line 83, where the timer period is computed from it. Change this one number and the timer follows automatically.

Line 13 · `#define WIN 256`

The window length in samples. This value is used in four places: it sizes the buffer on line 19, bounds the loops on lines 44 and 47, divides the mean on line 45, and divides the sum of squares on line 52. Naming it once means those five uses can never disagree. At 1 kHz, 256 samples is 256 ms of data, giving about four feature updates per second.

Line 15 · `static const char *TAG = "vib";`

The log tag printed with every `ESP_LOGI` line, identifying which module produced it. Read the declaration right to left: `TAG` is a pointer to `char`, the characters are `const` so they cannot be modified, and `static` makes the whole thing private to this `.c` file.

Line 16 · `static i2c_master_dev_handle_t mpu;`

The device handle for the sensor. A **handle** is an opaque pointer-like token: you never look inside it, you just pass it to the driver so it knows which device on which bus you mean. The `_t` suffix is C convention marking a type name. It sits at file scope because both `app_main` (which creates it) and `sampler_task` (which uses it) need access.

Lines 17-18 · `static SemaphoreHandle_t tick_sem; / win_ready;`

The two signalling objects that connect the pipeline's three stages, and the comments name their direction. `tick_sem` carries the signal timer → sampler; `win_ready` carries sampler → feature. They are declared here but not created until lines 75-76 — a handle declared is just an uninitialised pointer, and using one before it is created is a crash.

Lines 19-20 · `static float win_buf[WIN]; static int win_idx = 0;`

`win_buf` is the window: an array of 256 floats, which at 4 bytes each is 1 KB of RAM. Because it is `static` and file-scope with a non-zero-initialised sibling, it lives in the `.bss` section — in RAM, zeroed at boot, costing nothing in the flash image. `win_idx` tracks how many samples are currently in it.

WARNING**Notice what is not here: a mutex — and that this is a real race**

`win_buf` is written by the sampler and read by the feature task with **no lock at all**. Be honest about what that means rather than waving it away.

Why it works in practice. The sampler fills all 256 slots, resets the index, and only then signals. The feature task then has a full window period — 256 ms at 1 kHz — to finish reading before the sampler wraps back around to slot 0 and starts overwriting. Two passes over 256 floats takes microseconds. The margin is enormous, so you will never see this fail on the bench.

Why it is still a race. Nothing in the code *enforces* that margin. If the feature task were preempted for a long time, or you raised the sample rate, or you added work to the feature loop, the sampler would begin overwriting slots the feature task has not read yet — and the corruption would be silent, showing up as an occasional wrong RMS rather than a crash. The safety is a property of the *timing*, not of the *structure*, and timing-based safety is exactly the kind that stops holding when requirements change.

The two real fixes. A **double buffer** (ping-pong): the sampler fills buffer A while the feature task reads buffer B, then they swap, so the producer physically cannot touch what the consumer holds. Or a **queue** carrying whole windows, which makes the handoff explicit and bounded. The double buffer costs one extra kilobyte of RAM and removes the race entirely.

Say this accurately in an interview and it is a strength: *"there is a latent race on the shared window; it is safe by timing margin rather than by construction, and the correct fix is a double buffer."* Claiming it is simply safe is the answer that gets picked apart.

Lines 22-24 · `static void tick_cb(void *arg) { xSemaphoreGive(tick_sem); }`

The timer callback, and its brevity is the entire point. It takes the standard `void *arg` (a "pointer to anything," unused here) and does exactly one thing: `xSemaphoreGive(tick_sem)` raises the flag the sampler is blocked on.

What it deliberately does *not* do is the lesson. No I²C read, because `i2c_master_transmit_receive` **blocks** waiting on the bus, and blocking in a callback or interrupt context is illegal — there is no task there to put to sleep. No logging, no floating-point maths. While this callback runs it is delaying the next tick, so at 1 kHz you have a 1 ms budget and spending 0.4 ms here would burn 40% of it before the real work even starts. **TM4C bridge:** a bare-metal timer ISR sets a flag and clears the interrupt in `ICR`; identical division of labour, except here the waiting side can sleep instead of poll.

Line 26 · `static void sampler_task(void *arg)`

A FreeRTOS task function: returns `void`, takes one `void *`, and contains an infinite loop that never returns. A task function that falls off its end crashes the system; if a task must finish it calls `vTaskDelete(NULL)` instead.

Line 28 · `xSemaphoreTake(tick_sem, portMAX_DELAY);`

Where the sampler waits. **Take** decrements the semaphore's count, and if the count is already zero the calling task is **blocked** — moved out of the scheduler's ready list entirely, consuming no CPU — until someone gives it. The scheduler then wakes it immediately.

`portMAX_DELAY` is a constant meaning "wait forever, never time out." That is correct here because a missing tick would mean the timer itself died, which is not a recoverable condition worth writing code

around. In production code touching hardware that can genuinely stall, you would pass a real timeout in ticks and handle the failure. Knowing *when* an infinite wait is acceptable rather than lazy is part of the judgement being built.

Line 29 · `uint8_t reg = 0x3B, r[6];`

Two declarations in one statement. `reg` holds the register number to read from: **0x3B is ACCEL_XOUT_H**, the first of the six accelerometer data registers, and that number comes from the manufacturer's register map document. `r[6]` is the six-byte destination — six because the sensor stores three axes as three 16-bit values, laid out consecutively.

Line 30 · `i2c_master_transmit_receive(mpu, ®, 1, r, 6, 1000);`

One call performing the standard two-part I²C register read. First it **writes** one byte — the register address, passed as `&` because the driver wants a buffer pointer rather than a value — telling the sensor where you want to read from. Then, instead of releasing the bus, it issues a **repeated start** and **reads** six bytes into `r`. Holding the bus across both halves prevents another controller from interleaving and moving the sensor's internal register pointer underneath you.

The six arguments in order: device handle, write buffer, write size, read buffer, read size, timeout in milliseconds. Reading all six bytes in one transaction rather than three separate two-byte reads matters for correctness as well as speed — separate reads could straddle a sensor update and give you X from one sample and Z from the next.

Lines 31-33 · `int16_t ax = (int16_t)((r[0] << 8) | r[1]);`

Rebuilding each axis from its two bytes. Three separate things are happening.

The shift and the OR. The sensor transmits a high byte and a low byte separately. `r[0] << 8` shifts the high byte left by eight bit positions, moving it into the upper half of a 16-bit value; `| r[1]` then ORs the low byte into the lower half. This is **big-endian** ordering — most-significant byte transmitted first — and it is what the datasheet specifies for this part.

Integer promotion makes it safe. `r[0]` is a `uint8_t`, and C's rules automatically widen it to a 32-bit `int` before the shift is evaluated. So nothing is lost off the top; the intermediate result has plenty of room.

The `(int16_t)` cast is the part that matters. Acceleration can be negative, and `int16_t` is a *signed* 16-bit type, so the cast makes the top bit act as a sign bit in two's-complement representation. Without it, a value like `0xC000` would be read as 49152 instead of −16384, and a tilted or inverted board would report a huge positive number rather than a negative one. This single cast is the difference between working and silently wrong.

Line 34 · `float mag = sqrtf((float)ax*ax + ...) / 16384.0f;`

Three ideas in one line.

Why combine the axes at all. You care how much the object is shaking, not in which direction. The vector magnitude $\sqrt{x^2+y^2+z^2}$ collapses three signals into one number that does not depend on how the board happens to be oriented on the machine.

Why the casts. `ax` is an `int16_t`, so `ax*ax` would be evaluated in integer arithmetic, where the product of two values near the type's limit behaves in ways that are easy to get wrong. Casting to `float` first forces the whole expression into floating point.

Why 16384. At the default $\pm 2g$ full-scale range this sensor reports **16384 counts per g**, a figure from the sensitivity table in the manufacturer's product specification. Dividing by it converts raw counts into units of g. The `.0f` suffix makes the literal a `float`, forcing floating-point division — writing `/ 16384` would be integer division and would discard everything after the decimal point. Change the full-scale range via the `ACCEL_CONFIG` register (`0x1C`) and this divisor must change with it.

Line 35 · `win_buf[win_idx++] = mag;`

Stores the sample and advances the index. `win_idx++` is **post-increment**: the current value is used as the array index *first*, then `win_idx` is incremented. So the very first sample lands at index 0, which is what you want. Pre-increment (`++win_idx`) would skip index 0 and write one past the end on the final sample.

Line 36 · `if (win_idx >= WIN) { win_idx = 0; xSemaphoreGive(win_ready); }`

The second stage of the handoff. When 256 samples have accumulated, the index resets to 0 and `xSemaphoreGive(win_ready)` wakes the feature task — exactly once per window, so roughly four times a second rather than a thousand.

The ordering inside the braces is deliberate: the index is reset **before** the semaphore is given. That is what makes the no-lock argument in the box above hold — by the time the feature task can possibly run, the sampler has already finished with this window and moved on.

Line 40 · `static void feature_task(void *arg)`

The same task shape as the sampler, but it blocks on a different semaphore and therefore runs at a completely different rate. This is the payoff of the whole design: two pieces of code, each running at the rate its job actually requires, neither able to delay the other.

Line 42 · `xSemaphoreTake(win_ready, portMAX_DELAY);`

Sleeps until a full window is available. Between windows this task consumes zero CPU, which is why the expensive floating-point work it does next cannot interfere with the 1 kHz sampling path.

Lines 43-45 · the mean, first pass

`float mean = 0;` then a loop accumulating every sample, then `mean /= WIN`. Straightforward, but the reason it exists is not: the mean is the **DC component** of the signal. A board sitting still reads about 1.0 g because of gravity, and that constant offset tells you nothing about vibration. Computing the mean here lets the next pass subtract it out.

Lines 46-51 · the deviations, second pass

`float d = win_buf[i] - mean;` is one sample's deviation from the average — the AC part, with gravity removed. `ss += d * d;` accumulates the squared deviations; squaring makes negative and positive excursions count equally and weights large ones more heavily. `if (fabsf(d) > peak) peak = fabsf(d);` tracks the largest absolute deviation seen, where `fabsf` is floating-point absolute value.

Why two passes and not one. Every deviation is measured *from the mean*, and you cannot know the mean until you have seen every sample — so a single pass is impossible without a different formula. There *is* a one-pass formula using the sum and the sum of squares, and it is faster, but it loses numerical precision badly when the mean is large relative to the variation. That is exactly this case, since gravity dominates. The two-pass version is the numerically stable choice, and that is a good answer if an interviewer asks why you did it the slow way.

Line 52 · `float rms = sqrtf(ss / WIN);`

Root-mean-square: divide the accumulated squares by the count to get the mean square, then take the square root. Physically this is the "typical" amount of movement across the window, and it is the standard measure of vibration energy. Because the mean was subtracted first, this is specifically the **AC RMS** — a still board gives a value near zero, not near one.

Line 53 · `ESP_LOGI(TAG, "rms=%.4f peak=%.4f", rms, peak);`

Prints both features to four decimal places. `%.4f` is a float with four digits after the point; four is enough resolution to see a still board's small non-zero noise floor, which is itself useful information.

Lines 59-63 · `i2c_master_bus_config_t bus_cfg = { ... }`

Configuring the I²C bus itself. `.clk_source = I2C_CLK_SRC_DEFAULT` lets the driver pick the clock the bus timing is derived from. `.i2c_port = I2C_NUM_0` selects one of the chip's two hardware I²C units. `.sda_io_num` and `.scl_io_num` assign the two wires to GPIO8 and GPIO9 — possible because the ESP32-S3's GPIO matrix can route almost any peripheral signal to almost any pin.

`.glitch_ignore_cnt = 7` tells the hardware to ignore pulses shorter than seven clock cycles, which is free noise rejection and genuinely helpful on breadboard wiring. `.flags.enable_internal_pullup = true` is a **nested field** — `flags` is itself a small struct inside the config, reached with a second dot — and it switches on the chip's internal pull-up resistors. Those matter because I²C lines are **open-drain**: devices can only pull a line low, never drive it high, so a pull-up is what returns the line to high when nobody is pulling. Your breakout board also carries its own pull-ups; having both simply puts two resistors in parallel, which is harmless at this speed.

Lines 64-65 · `i2c_master_bus_handle_t bus; i2c_new_master_bus(&bus_cfg, &bus);`

Creates the bus and writes a handle into `bus`. Note there are now **two** handles in this program: one for the bus and one for the device on it. That separation is what allows several chips to share the same two wires, each registered separately.

Worth noticing: this call returns an `esp_err_t` which this listing discards. In your own copy, wrap it in `ESP_ERROR_CHECK()` so a configuration mistake aborts loudly instead of leaving you with a bus that silently does nothing.

Lines 66-69 · `i2c_device_config_t dev_cfg = { ... }`

Describing the specific chip on that bus. `.dev_addr_length = I2C_ADDR_BIT_LEN_7` says this device uses 7-bit addressing, which is the normal case; a 10-bit mode exists for buses needing more than 128 devices. `.device_address = MPU_ADDR` is the `0x68` from line 11.

`.scl_speed_hz = 400000` sets the clock to 400 kHz, I²C "fast mode" and this sensor's limit. This is a **per-device** setting, so a slower chip sharing the bus could be registered at 100 kHz and the driver would adjust per transaction. It also matters directly for this lab: at 1 kHz sampling you have a 1 ms

budget per sample, and a bus accidentally left at 100 kHz makes each six-byte read four times slower, which is enough to start dropping samples.

Line 70 · `i2c_master_bus_add_device(bus, &dev_cfg, &mpu);`

Registers the device on the bus and fills in the `mpu` handle declared on line 16. From this point on, `mpu` is the sensor as far as the rest of the program is concerned.

Lines 71-72 · `uint8_t wake[2] = { 0x6B, 0x00 }; i2c_master_transmit(mpu, wake, 2, 1000);`

Waking the sensor. **0x6B is the PWR_MGMT_1 register** and **0x00** is the value being written to it. The two bytes are packed into an array because the I²C convention for a register write is **first byte says which register, second byte is the data**, sent as one transaction.

Why this is necessary: the MPU-6050 **boots asleep** — bit 6 of PWR_MGMT_1 is the SLEEP bit and its power-on value is 1. Writing **0x00** clears every bit in that register, including SLEEP, and also selects the internal oscillator as the clock source. Omit this line on a 6050 and every reading comes back exactly zero, which is the single most common first-time failure with this part. On an MPU-6500 the reset state differs slightly, but writing zero still leaves it awake and internally clocked, so the same line is correct for both.

Note this listing does the wake inline rather than through a helper function, and it does not check the return value — both worth tightening in your own version.

Lines 75-76 · `tick_sem = xSemaphoreCreateBinary(); win_ready = xSemaphoreCreateBinary();`

Allocates the two semaphores declared on lines 17-18. A **binary** semaphore's count saturates at 1 — it is a flag that is either raised or not, and two gives before a take are indistinguishable from one. That has a real consequence here: if the sampler ever takes longer than 1 ms, a tick is silently dropped rather than queued up. For a sampler that is usually the *right* behaviour (better to skip than fall permanently behind), but it is a design decision worth being able to defend, not an accident.

Both are created **empty**, which is what you want — the sampler should block on line 28 until the first real tick arrives.

Lines 78-79 · `xTaskCreatePinnedToCore(sampler_task, "samp", 4096, NULL, 6, NULL, 1);`

Seven arguments: the function to run, a short name for debugging, the stack size **in bytes** (ESP-IDF counts bytes; upstream FreeRTOS counts words, which is a real portability trap), the argument passed to the task, the priority, somewhere to store a task handle, and — new in this call — **the core number**.

Why core 1. The chip has two cores. Wi-Fi and Bluetooth run their own time-critical stacks on core 0, and radio activity is bursty. Pinning both of these tasks to core 1 keeps your deterministic sampling away from that interference. It costs nothing now and prevents mysterious jitter later when you add networking.

Why priority 6 for the sampler and 5 for the feature task. If both become ready at the same instant, the higher number runs first. The sampler has a hard 1 ms deadline; the feature task can be several milliseconds late with no consequence at all. Reversing these would let the slow reporting task preempt the sampler and put holes back in your data. The general principle is **rate-monotonic**: the more frequent and more deadline-sensitive the task, the higher its priority.

The 4096-byte stacks are generous for this work. §7.7 shows you how to measure what they actually use with `uxTaskGetStackHighWaterMark` rather than guessing.

Lines 80-82 · `esp_timer_create_args_t a = { .callback = &tick_cb, .name = "fs" }; ...`
`esp_timer_create(&a, &h);`

`.callback = &tick_cb` passes the address of the function to call. In C the `&` on a function name is optional, but writing it makes the intent explicit. `.name` appears in diagnostic output. `esp_timer_create(&a, &h)` then builds the timer and writes its handle into `h` — the same configure-then-create pattern as the I²C bus and device above.

Line 83 · `esp_timer_start_periodic(h, 1000000 / FS_HZ);`

Starts the timer. The period argument is in **microseconds**, which is the detail to verify in the documentation rather than assume — guessing milliseconds would make the sample rate wrong by a factor of a thousand while the code ran perfectly.

`1000000 / FS_HZ` is 1,000,000 microseconds per second divided by 1000 Hz, giving a 1000 μ s period. Writing it as an expression rather than the literal `1000` means changing `FS_HZ` on line 12 automatically retunes the timer, so the two can never silently disagree. Note this is integer division, which is fine for rates that divide evenly into a million.

Line 84 · `}`

`app_main` returns here, and the program keeps running. The two tasks and the timer are independent entities owned by the system, not things called from this function. **TM4C bridge:** on the TM4C, `main()` returning would have been catastrophic because it *was* the program. Here `app_main` is a launcher that is allowed to finish.

7.6 Docs & references

DOCS

Trace every line to its source

`xSemaphoreCreateBinary`, `xSemaphoreGive`, `xSemaphoreTake` — FreeRTOS API, semaphores. Note the timeout is in **ticks**, and read what `portMAX_DELAY` means.

`xTaskCreatePinnedToCore` — ESP-IDF FreeRTOS additions. Confirm the argument order and that stack depth is in **bytes** here, unlike upstream FreeRTOS which counts words.

`esp_timer_start_periodic` — System → ESP Timer. Microseconds, which is why the period is written `1000000 / FS_HZ`.

`i2c_master_transmit_receive` — Peripherals → I2C. Argument order: device handle, write buffer, write size, read buffer, read size, timeout in ms.

`sqrtf`, `fabsf` — `<math.h>`. Note the `f` suffix: these are the `float` versions, and using the `double` versions on a 32-bit MCU is needlessly slow.

`0x3B` — the `ACCEL_XOUT_H` register, from the **MPU-6500 Register Map and Descriptions**. 16384 — counts per g at the default ± 2 g range, from the accelerometer sensitivity table in the **MPU-6500 Product Specification**. Two separate PDFs; download both and confirm each number yourself rather than trusting this document.

7.7 Verify, troubleshoot, your turn

The log alone is weak evidence. Three checks, in increasing order of rigour:

1. **Physical sanity.** Board still → RMS and peak both near zero. Tap the desk → both jump. Hold it against something humming → RMS rises and stays up. If RMS reads near 1.0 at rest, you are not subtracting the mean and gravity is leaking into your feature.
2. **Rate check.** Windows should print about 4× per second ($1000 \text{ Hz} \div 256$). Noticeably slower means samples are being dropped — the sampler is missing ticks.
3. **Timing check with the analyzer.** This is the real one. Toggle a spare GPIO high at the top of the sampler and low at the bottom, then capture it. You get a direct picture of **how long each sample actually takes** and whether the period is a rock-steady 1 ms. The high time is your execution time; the period is your jitter. If high time approaches 1 ms you are near overrun. This measurement — instrumenting code with a GPIO and watching it on an analyzer — is a standard professional technique and a great thing to have done.

Troubleshoot

Symptom	Cause and fix
No feature lines at all	The timer never started, or the sampler is blocked on I ² C. Check WHO_AM_I still answers.
RMS sits near 1.0 at rest	You are not subtracting the mean, so gravity is leaking into the feature.
Windows arrive much slower than 4 Hz	Samples are being dropped — the sampler is missing ticks. Measure with the GPIO trick.
Guru Meditation in the sampler	Stack too small for the floating-point work. Raise it, then measure with the high-water mark.
Values freeze after a while	The I ² C transaction is failing and the return code is unchecked. Wrap it and log failures.

Your turn

The main manual's extension is open-ended. Three concrete ones, ordered by interview value:

1. **Instrument and measure.** Do check 3 above. Write down your measured execution time and jitter in `debug_log.md`. Being able to say "my sampling loop ran in 310 μs with under 20 μs of jitter at 1 kHz, measured on a logic analyzer" is a concrete, credible engineering claim.
2. **Deliberately break it.** Add a 5 ms delay inside the sampler so it cannot keep up. Watch what happens: the binary semaphore saturates, ticks are dropped, the window rate falls. Then reason about which fix is right — a counting semaphore that queues the backlog, or accepting the drop. This is a designed failure and a strong war story.
3. **Add the deadline check.** Have the timer callback increment a counter and the sampler decrement it; if it ever exceeds one, you missed a deadline. Log it. This is real-time monitoring, exactly what safety-critical firmware does.

Errata and corrections

Defects confirmed against the hardware and the vendor documentation, with the evidence that established each one. They are printed rather than quietly patched because how an error was caught is usually more instructive than the error.

Additional corrections

The race is left optional. §7.5 correctly names the shared-window race but leaves the double buffer as an exercise. It should be mandatory. Separately, the 1 kHz tick uses a **binary** semaphore, whose count saturates at one — so two ticks arriving before the sampler runs are indistinguishable from one, and the sample is silently dropped. Count missed ticks rather than losing them invisibly.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 7.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07 ←	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.